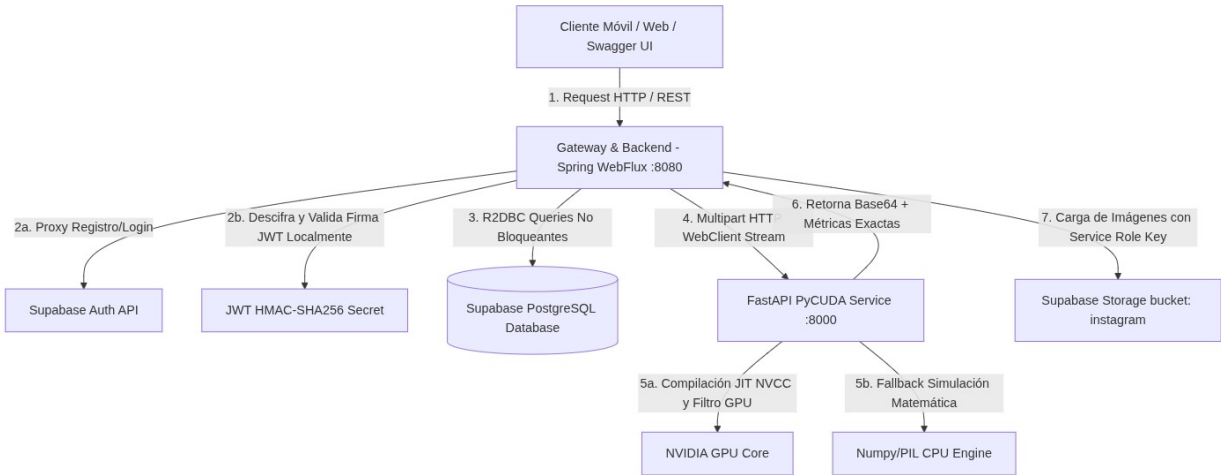


	VICERRECTORADO DOCENTE	Código: GUIA-PRL-001
	CONSEJO ACADÉMICO	Aprobación: 2016/04/06
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		

	PRÁCTICA DE LABORATORIO
CARRERA: COMPUTACIÓN	ASIGNATURA: COMPUTACIÓN PARALELA
NRO. PRÁCTICA: -	TÍTULO PRÁCTICA: UPS GLAM 3.0
OBJETIVO ALCANZADO: El propósito del proyecto es que los estudiantes desarrollen una aplicación real e integrada, aplicando los conceptos fundamentales de Computación Paralela en un escenario práctico de software.	
ACTIVIDADES DESARROLLADAS	
1. Diseño de la Arquitectura de Microservicios: Definición de la comunicación entre el cliente móvil, el backend reactivo y el motor de procesamiento GPU mediante Docker.  <pre> graph TD Client[Cliente Móvil / Web / Swagger UI] -- "1. Request HTTP / REST" --> Gateway[Gateway & Backend - Spring WebFlux :8080] Gateway -- "2a. Proxy Registro/Login" --> SupabaseAuth[Supabase Auth API] Gateway -- "2b. Descifra y Valida Firma JWT Localmente" --> JWTSecret[JWT HMAC-SHA256 Secret] Gateway -- "3. R2DBC Queries No Bloqueantes" --> SupabaseDB[(Supabase PostgreSQL Database)] Gateway -- "4. Multipart HTTP WebClient Stream" --> FastAPI[FastAPI PyCUDA Service :8000] FastAPI -- "5a. Compilación JIT NVCC y Filtro GPU" --> GPU[NVIDIA GPU Core] FastAPI -- "5b. Fallback Simulación Matemática" --> CPU[Numpy/PIL CPU Engine] Gateway -- "6. Retorna Base64 + Métricas Exactas" --> FastAPI Gateway -- "7. Carga de Imágenes con Service Role Key" --> SupabaseStorage[Supabase Storage bucket: instagram] </pre>	

	VICERRECTORADO DOCENTE	Código: GUIA-PRL-001
	CONSEJO ACADÉMICO	Aprobación: 2016/04/06
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		

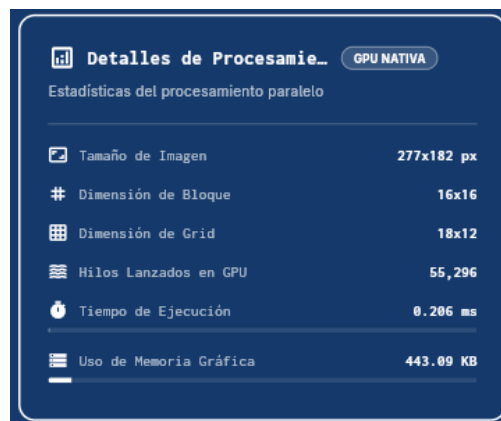
2. Implementación de Kernels CUDA en PyCUDA: Desarrollo de algoritmos de convolución paralela para filtros como Box Blur, Sobel, Cartooning y filtros de identidad institucional UPS.



3. Desarrollo del Backend Reactivo: Uso de Spring WebFlux y R2DBC para gestionar publicaciones, comentarios y likes sin bloquear los hilos del servidor

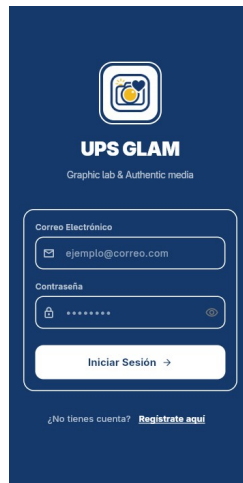
4. Integración de Servicios Cloud (Supabase): Configuración de Supabase Auth para seguridad con JWT, Storage para imágenes y PostgreSQL para la persistencia de datos relacionales.

5. Sistema de Telemetría de Hardware: Implementación de captura de métricas en tiempo real (uso de VRAM, dimensiones de bloque/grid, hilos totales y tiempo de ejecución) desde el motor CUDA hacia la base de datos.



	VICERRECTORADO DOCENTE	Código: GUIA-PRL-001
	CONSEJO ACADÉMICO	Aprobación: 2016/04/06
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		

6. Desarrollo de Aplicación Móvil en Flutter: Creación de una UI para los usuarios finales



7. Orquestación con Docker Compose: Creación de imágenes Docker y archivos de orquestación para el despliegue automático del ecosistema de servicios.

```

1 version: '3.8'
2
3 services:
4   # 1. PyCUDA Image Processing Microservice
5   cuda-service:
6     build:
7       context: ./cuda-service
8       dockerfile: Dockerfile
9     container_name: cuda-service
10    ports:
11      - "8000:8000"
12    environment:
13      - HOST=0.0.0.0
14      - PORT=8000
15    networks:
16      - instagram-net
17    restart: unless-stopped
18    deploy:
19      resources:
20        reservations:
21          devices:
22            - driver: nvidia
23              count: all
24              capabilities: [ gpu ]
25
26   # 2. Spring WebFlux Integration Backend
27   backend-spring:
28     build:
29       context: ./backend-spring
30       dockerfile: Dockerfile
31     container_name: backend-spring
32     ports:
33       - "8080:8080"
34     depends_on:
35       - cuda-service
36     env_file:
37       - .env
38     environment:
39       - CUDA_SERVICE_URL=http://cuda-service:8000
40     networks:
41       - instagram-net
42     volumes:
43       - ./backend-spring/logs:/app/logs
44     restart: unless-stopped
45
46 networks:
47   instagram-net:
48     driver: bridge
49

```

	VICERRECTORADO DOCENTE	Código: GUIA-PRL-001
	CONSEJO ACADÉMICO	Aprobación: 2016/04/06
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		

RESULTADO(S) OBTENIDO(S):

- **Integración Completa:** Se obtuvo un sistema funcional donde una imagen cargada desde el móvil es procesada en milisegundos por la GPU y publicada globalmente en tiempo real.
- **Eficiencia en Procesamiento:** Los filtros aplicados mediante kernels CUDA mostraron una reducción drástica en el tiempo de ejecución comparado con implementaciones secuenciales tradicionales.
- **Persistencia y Seguridad:** Registro exitoso de usuarios y almacenamiento seguro de métricas técnicas y multimedia en la nube de Supabase.
- **Telemetría Transparente:** La aplicación muestra con precisión los recursos de hardware utilizados, permitiendo verificar el paralelismo aplicado
-

CONCLUSIONES:

- El uso de Computación Paralela (CUDA) es esencial para aplicaciones multimedia modernas, permitiendo que operaciones complejas sobre píxeles se ejecuten simultáneamente en miles de núcleos.
- La implementación de programación reactiva demostró ser el complemento obligatorio para el procesamiento paralelo en red. Evitar el modelo de hilo por petición (*thread-per-request*) permite que el sistema espere la respuesta de la GPU sin bloquear los recursos de RAM e I/O del servidor.
- El uso de PostgreSQL estructuró correctamente la integridad de datos relacionales (usuarios, posts, multimedia). El uso de Supabase como plataforma gestionada (DBaaS) simplificó la infraestructura, pero el verdadero logro técnico fue acceder a ella mediante R2DBC, manteniendo la naturaleza reactiva de toda la aplicación desde el cliente hasta la base de datos.
- La contenedorización resolvió las discrepancias de dependencias operativas, especialmente críticas en proyectos que involucran librerías nativas del sistema operativo para el manejo de la tarjeta gráfica y drivers. Garantizó que la arquitectura sea replicable, escalable y tolerante a fallos independientemente del entorno de ejecución local.

RECOMENDACIONES:

- Para filtros de convoluciones grandes, se recomienda implementar el uso de memoria compartida de CUDA. Esto permitiría reducir el tráfico hacia la memoria global, disminuyendo los cuellos de botella y mejorando significativamente el throughput del sistema.
- Para una puesta en producción masiva, se recomienda balancear el tráfico entre varias instancias del microservicio `cuda-service` distribuidas en nodos con diferentes GPUs, aprovechando la naturaleza desacoplada de la arquitectura Docker.
- Para futuras versiones, se sugiere implementar filtros de procesamiento de video en tiempo real aprovechando la base reactiva y paralela ya establecida

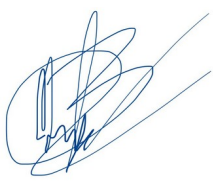
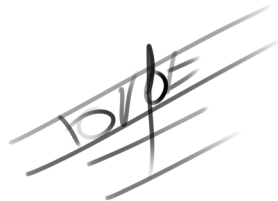
	VICERRECTORADO DOCENTE	Código: GUIA-PRL-001
	CONSEJO ACADÉMICO	Aprobación: 2016/04/06
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		

REFLEXION FINAL:

UPSGlam es una representación entre la teoría de la computación paralela y su aplicación en productos comerciales reales. La capacidad de observar métricas de hardware directamente en una red social educativa fomenta una comprensión profunda de cómo los hilos y bloques de CUDA impactan en la experiencia del usuario final. La integración de tecnologías reactivas y cloud asegura que el proyecto no sea solo un ejercicio académico, sino una solución escalable y moderna.

ANEXOS Y CÓDIGO FUENTE:

El código fuente del proyecto, abarcando la orquestación de contenedores, la aplicación móvil y los *kernels* de PyCUDA, se encuentra en el siguiente repositorio: <https://github.com/KarenS22/UPS-Glam.git>

Nombre de estudiante:	Firma de estudiante:
Mateo Barzalio	
Jorge Cueva	
Karen Quito	